

Assessment Cover Sheet

This Assessment Cover Sheet is only to be attached to hard copy submission of assessments.



ASSESSMENT DETAILS

Unit title	COMPUTING TECHNOLOGY INNOVATION PROJECT	Tutorial /Lab Group	-	Office use only
Unit code	COS 30049	Due date	27/3/2026	
Name of lecturer/tutor	Ts. Dr. Lee Su han			
Assignment title	Project Proposal			Faculty or school date stamp

STUDENT(S) DETAILS

	Student Name(s)	Student ID Number(s)
(1)	TERENCE CHIN SENG WONG	104404059
(2)	TAMMY RU XIU TAY	104405861
(3)	MIAW FONG LIM	104406013
(4)	EMILY HUI XIU JONG	102788832
(5)	ZI YANG LIM	102774129
(6)	YOUNG FUNG LAU	102774585
(7)	BRIAN JIA HUA LIEW	104381288

DECLARATION AND STATEMENT OF AUTHORSHIP

1. I/we have not impersonated, or allowed myself/ourselves to be impersonated by any person for the purposes of this assessment.
2. This assessment is my/our original work and no part of it has been copied from any other source except where due acknowledgement is made.
3. No part of this assessment has been written for me/us by any other person except where such collaboration has been authorised by the lecturer/tutor concerned.
4. I/we have not previously submitted this work for this or any other course/unit.
5. I/we give permission for my/our assessment response to be reproduced, communicated, compared and archived for plagiarism detection, benchmarking or educational purposes.

I/we understand that:

6. Plagiarism is the presentation of the work, idea or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to exclusion from the University. Plagiarised material can be drawn from, and presented in, written, graphic and visual form, including electronic data and oral presentations. Plagiarism occurs when the origin of the material used is not appropriately cited.

Student signature/s

I/we declare that I/we have read and understood the declaration and statement of authorship.

(1) WONG	(5) LIM
(2) TAY	(6) LAU
(3) LIM	(7) LIEW
(4) JONG	(8)

Further information relating to the penalties for plagiarism, which range from a formal caution to expulsion from the University is contained on the Current Students website at <https://www.swinburne.edu.my/current-students/manage-course/exams-results-assessment>

Copies of this form can be downloaded from the Student Forms web page at <https://www.swinburne.edu.my/current-students/manage-course/exams-results-assessment/how-to-submit-work.php>

Table of Contents

Executive Summary	3
Background / Problem Description	4
Scope	5
Stakeholders.....	6
Deliverables and schedule.....	8
Initial Release Schedule – Product Backlog with Subtasks.....	11
Solution Direction	17
Solution 1 — Custom-Built Next.js / Supabase Fullstack PWA with On-Device AI	17
Solution 2 — Off-the-Shelf LMS (Moodle / TalentLMS) with Cloud-Based AI Integration	17
KoST Gap Analysis	18
Proposed Solution Direction and Justification.....	22
Discarded potential alternative solution.....	23
AI Monitoring — Edge Device Feasibility Evaluation.....	23
High-Level System Architecture	26
Quality Management.....	32
Resources	34
Project Team	36

Project Proposal¹ : Digital Park Guide Training Platform for Sarawak's National Parks and Nature Reserves

EXECUTIVE SUMMARY

The Sarawak Forestry Corporation (SFC) manages 67 Totally Protected Areas across Sarawak, supported by approximately 200–250 active licensed park guides and several hundred field rangers. Current training programmes are physically constrained and instructor-dependent, resulting in a guide workforce that remains below its target capacity and lacks consistent access to up-to-date conservation knowledge. Simultaneously, the absence of any real-time field monitoring mechanism leaves SFC's protected areas dependent on reactive rather than preventive enforcement.

This proposal describes the development of an integrated Digital Park Guide Training Platform — a custom-built web and mobile application that replaces the current training model with a flexible, self-paced e-learning system, while embedding an AI-powered monitoring capability that detects and records conservation violations during guided tours in real time.

The platform is built on Next.js and Supabase, deployed via Vercel, and designed to operate in the low-connectivity environments characteristic of Sarawak's remote parks. It will be delivered across two three-week sprints by a team of eight developers, producing a fully functional staging deployment, complete technical documentation, a security assessment, and user manuals for both field staff and administrators.

¹ This document is by no means a “full project proposal”. It has been simplified and customized for the purposes of MSP teaching. The full project proposal includes many other sections which have not been discussed during the first few weeks of MSP teaching.

BACKGROUND / PROBLEM DESCRIPTION

Sarawak's national parks and nature reserves are among the most biodiverse ecosystems in Southeast Asia, and the park guides and rangers who steward these environments play a critical role in shaping both visitor experience and conservation outcomes. SFC recognises that the growing demands of eco-tourism require a more structured, scalable approach to guide and ranger professional development.

Sarawak Forestry Corporation (SFC) currently manages 67 gazetted Totally Protected Areas (TPAs) across Sarawak, comprising 47 national parks, 15 nature reserves, and 5 wildlife sanctuaries. Supporting these protected areas is a workforce of approximately 200–250 active licensed park guides — against a government target of at least 300 — and an estimated several hundred field rangers, including 250 Wildlife Guardian Rangers recruited from army veterans and local communities to assist with enforcement and patrol duties. Despite this considerable operational scale, the training infrastructure supporting these personnel remains predominantly instructor-led and physically centralised, placing significant logistical burdens on those expected to participate. Guides sometimes have to travel to central sites, take long leaves from their job, and forgo income during training periods, a burden that falls particularly hard on guides stationed at remote park locations. The existing constraints have resulted in reduced enrolment figures, incomplete training initiatives, and an overall decrease in the professional readiness of the guiding workforce. Furthermore, the unchanging nature of these programs leads to infrequent updates on the latest developments in conservation science, park regulations, and eco-tourism best practices, resulting in gaps in knowledge that could threaten visitor safety and environmental protection.

The SFC has concluded that guided field excursions necessitate enhanced oversight alongside training efforts. Without a reliable, scalable mechanism to monitor guide conduct and visitor behaviour in real time, enforcement of park regulations remains reactive rather than preventive — leaving protected flora and fauna vulnerable to prohibited activities that may go undetected or undocumented.

This proposal suggests building an integrated Digital Park Guide Training Platform to deal with these connected problems. The platform will provide modular, self-paced training content accessible via both web browser and mobile device allowing guides to improve their skills flexibly without interfering with field operations. Complementing this, an AI monitoring system leveraging device cameras and real-time computer vision with the detection model served from the cloud and cached on-device via the PWA service worker, enabling offline inference without dependency on a persistent server connection, proactively detect abnormal or non-compliant activities during guided tours, trigger automated alerts to park authorities, and securely archive video evidence for accountability and future training reference.

This initiative is to create a smarter, more connected park management system that uses new technology to improve professional standards, make conservation enforcement stronger, and make Sarawak's natural heritage sites world-class eco-tourism destinations.

SCOPE

This project delivers an integrated Digital Park Guide Training Platform for the Sarawak Forestry Corporation (SFC), addressing two core operational gaps that cannot be resolved through existing off-the-shelf solutions.

The first gap is training accessibility. SFC's current instructor-led model requires guides and rangers to attend centralised sessions, creating barriers of travel, income loss, and scheduling conflict that have suppressed enrolment and left the active guide pool below its target capacity. A digital platform eliminates these barriers by delivering training through a web browser and installable mobile application, allowing field staff to complete certified training modules at their own pace without leaving their assigned park.

The second gap is field oversight. There is currently no systematic mechanism for monitoring guide conduct or detecting conservation violations during guided tours. The platform addresses this through an on-device AI monitoring system that operates independently of internet connectivity — a deliberate design decision given the unreliable network conditions across Sarawak's remote park environments. The platform delivers three functional areas stated below:

- Digital Training & Certification Management

A modular e-learning system supporting two certification tracks: Guide Training (focused on biodiversity, habitats, and tour facilitation) and Ranger Training (focused on park management and conservation operations). Each certificate is park-specific and non-transferable across TPAs, in line with SFC's operational structure. The system manages the full certification lifecycle from application submission through HoD review, interview scheduling, payment processing, and badge issuance.

- Multilingual content delivery

The platform supports multilingual content delivery via the Google Translate API, allowing training modules, notifications, and interface elements to be served in the user's preferred language — improving accessibility for guides and rangers across Sarawak's linguistically diverse communities.

- AI-Based Abnormal Activity Monitoring

An AI monitoring system using a pre-trained computer vision model served from the server and cached on-device via the PWA service worker. Once cached, the model runs inference locally against the device camera feed, enabling real-time detection of conservation violations without requiring an active internet connection to detect prohibited activities during guided tours in real time. Evidence is captured, compressed, and stored with full metadata for review and reporting by authorised personnel.

- Role-Based User Management

The platform supports a defined access hierarchy. SFC Management, HoD, Rangers (SFC operational staff responsible for park management and enforcement), Senior Guide, and Guide with data access scoped to each role at the database level, ensuring guides see only their own records and seniors see only their assigned groups.

What is explicitly out of scope for this delivery:

- i. Native iOS or Android application — the mobile experience is delivered via a Progressive Web App (PWA), which provides equivalent accessibility including home screen installation, offline caching, and push notifications without requiring App Store distribution or a separate codebase
- ii. Cloud-based AI inference — while the model file is hosted on the server and downloaded on first PWA load, all inference is executed locally on the device using the service-worker-cached model. No video frames are transmitted to a server for processing, preserving offline functionality.
- iii. VR/AR training simulations — identified as optional in the project brief and excluded from the current scope of delivery
- iv. Live payment processing — Stripe integration operates in sandbox mode for demonstration purposes; production payment activation is subject to a separate go-live arrangement
- v. Production environment deployment — the platform is developed and demonstrated in a staging environment; production infrastructure provisioning is outside the current engagement

STAKEHOLDERS

1. Dr Lee Su Han

Client Representative & Project Supervisor Dr Lee represents the Sarawak Forestry Corporation's interests in this engagement and serves as the academic supervisor overseeing project delivery. Her interest is in ensuring the platform addresses SFC's operational requirements — standardised training across all TPAs, regulation enforcement through AI monitoring, and a certification pipeline that is manageable by HoD-level administrators. She holds final approval authority over scope decisions and deliverable acceptance.

2. HoD / Administrators

Platform Operators Internal SFC staff responsible for day-to-day platform management. Their interest is in tools that allow them to efficiently manage training content, review certification applications, conduct interviews, issue badges, and review AI-flagged evidence without requiring technical expertise.

3. Park Guides

Field staff whose primary platform interaction is training, certification, and badge management. Their interest is in a system accessible from mobile devices that allows flexible, self-paced learning without disrupting field operations.

4. Public Users

Prospective Applicants Members of the public who may browse SFC announcements and training programme briefs before deciding to apply as a guide or ranger. Their interaction with the platform is read-only and requires no account creation.

5. Development Team

Project Delivery The student team responsible for designing, building, and delivering the platform within the agreed two-sprint timeline. The team's interest is in well-defined requirements, a clearly scoped engagement, and producing a technically sound system that meets the client's objectives.

The following diagram illustrates the role hierarchy within the platform, showing the reporting structure from SFC Management down to field-level guides and rangers.

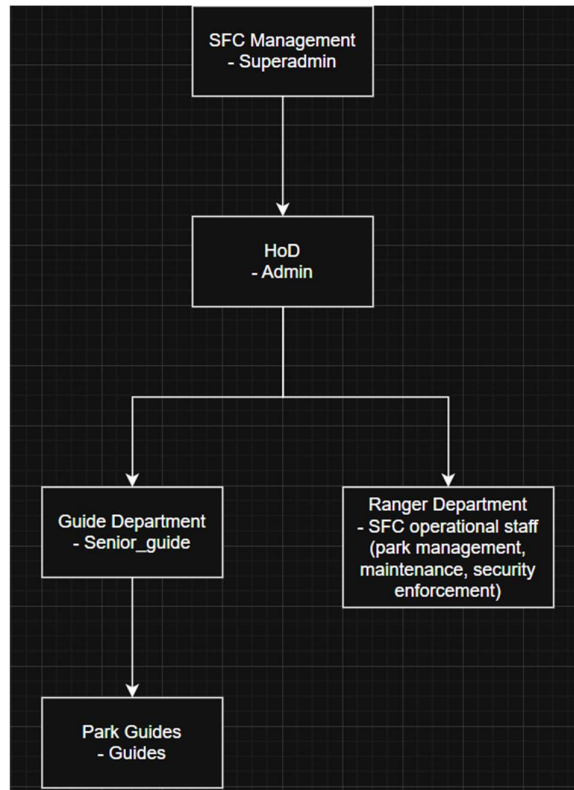


Figure 1 - Platform Role Hierarchy

DELIVERABLES AND SCHEDULE

1. Fully Functional Web Application

The core deliverable of this project is a complete Digital Park Guide Training Platform built using Next.js as the fullstack framework. This includes all frontend interfaces and backend API routes, role-based access across all user types (SFC Management, HoD, Senior Guide/Ranger, Guide/Ranger, and Public), and a Progressive Web App (PWA) configuration allowing mobile installation without App Store distribution. The application will be deployed to a staging environment via Vercel.

Acceptance Criteria:

- i. All six user roles (SFC Management, HoD, Senior Guide, Senior Ranger, Guide, Ranger) can authenticate successfully via both Google OAuth and OTP, and are presented with role-appropriate dashboards with no access to unauthorised routes
- ii. A guide or ranger can complete the full certification lifecycle end-to-end, from training module completion and quiz submission through to badge receipt via email, within a single user session without manual intervention from the development team
- iii. The platform is installable as a PWA on a mobile device and demonstrates offline access to previously loaded training content when network connectivity is disabled
- iv. Role-based data isolation is verified, a guide account cannot retrieve records belonging to another guide, and a senior can only access records belonging to their assigned group, confirmed through both UI testing and direct Supabase RLS policy verification
- v. The application passes deployment to Vercel staging with all environment variables configured, and all primary user flows are functional on the staging URL without local development dependencies

2. AI Monitoring System

An on-device abnormal activity detection module integrated into the platform, utilising a pre-trained computer vision model hosted on the server and cached on-device via the PWA service worker, with live device camera feed as input. This includes the full evidence pipeline, real-time detection, on-device alert, H.264 video compression via FFmpeg.wasm, upload to Supabase Storage, and PostgreSQL metadata logging — as well as offline queuing and background sync for low-connectivity environments.

Acceptance Criteria:

- i. The pre-trained model is successfully served from the server, downloaded, and cached by the service worker on first PWA initialisation. Following caching, the model must load and begin processing camera frames within 5 seconds of a monitoring session being started, without requiring an active network connection.
- ii. Upon detection, a timestamped evidence clip is captured, compressed to H.264 via FFmpeg.wasm, and uploaded to Supabase Storage with full metadata (guide ID, activity type, confidence score, timestamp) inserted into PostgreSQL — all within 60 seconds of the detection event on a network-connected device
- iii. Offline capability is demonstrated end-to-end: with network disabled after initial PWA load, (a) the model loads from service worker cache, (b) a detection event is triggered via device camera, (c) the evidence clip is queued locally, and (d) upon restoring connectivity, the queued clip syncs automatically to Supabase Storage without manual intervention.
- iv. The on-device alert (sound and visual) fires within 3 seconds of a detection event regardless of network status, confirmed through testing in both online and offline conditions
- v. All stored evidence clips are accessible to HoD via the admin dashboard, and Senior Guides/Rangers can only view clips associated with their assigned group — verified through role-based access testing

3. Database & Backend Infrastructure

All Supabase configurations required to support the platform, including the full PostgreSQL schema with Row Level Security (RLS) policies, Edge Functions, Storage bucket setup and access policies, and forward-only reversible database migration scripts. Seed data covering sample training modules, test user accounts, and sample evidence clips will also be provided to support demonstration and testing.

4. Third-Party Integrations

Working integrations with all external service providers used by the platform. Stripe (sandbox payment processing for certification fees), Twilio (SMS notifications), Google Translate API, Supabase AUTH (Google OAuth) and Web Push API with VAPID keys (PWA push notifications).

5. Technical Documentation

A complete set of technical documents to support understanding, maintenance, and future development of the system, including system architecture overview, all system flow diagrams, API documentation, and a setup and deployment guide covering environment configuration and local development setup.

6. Security Assessment

A full application vulnerability assessment conducted in accordance with the Secure Software Development Lifecycle (SSDLC), including the testing plan, tools used, findings report, implemented remediation, and a reassessment report confirming that identified vulnerabilities have been resolved.

7. User Manuals

Two separate user manuals written for non-technical audiences — one covering the Guide/Ranger perspective and one covering the HoD/Admin perspective — documenting all key workflows and system interactions.

INITIAL RELEASE SCHEDULE – PRODUCT BACKLOG WITH SUBTASKS

The product backlog is organised into 6 tracks (A–F) proceeding concurrently across two sprints. Each backlog item is broken down into subtasks representing discrete development tasks. Total: 118 subtasks across 29 backlog items. Tracks A–C and item D1 are Sprint 1 priorities; Tracks D2–D7, E, and F are Sprint 2.

No.	Item	Subtask	Functionality	Dep	BV	Sprint
Track A — Authentication & Access Control						
A1	User Registration & Authentication (OAuth + OTP)	A1.1 — Google OAuth provider setup	Configure Google OAuth provider in Supabase Auth and register OAuth credentials in Google Cloud Console	NA	10	Sprint 1
		A1.2 — OAuth sign-in flow (Next.js)	Implement Google sign-in button and Supabase OAuth redirect handler with session persistence			
		A1.3 — OTP email/phone verification flow	Implement OTP request form, Supabase Auth OTP trigger, and OTP input verification UI with expiry handling			
		A1.4 — JWT session management	Configure Supabase session listener, token refresh logic, and secure cookie storage for authenticated sessions			
		A1.5 — Auth error handling & user feedback	Implement error states for invalid OTP, expired session, OAuth failure, and duplicate account with user-facing messages			
A2	Role-Based Access Control (RBAC)	A2.1 — Role enum & permission matrix	Define role enum (SFC_MANAGEMENT, HOD, RANGER, SENIOR_GUIDE, GUIDE, PUBLIC) and document permission matrix per role	A1	10	Sprint 1
		A2.2 — PostgreSQL RLS policies	Write and apply Row Level Security policies on all tables scoped to each role			
		A2.3 — Next.js middleware route protection	Implement Next.js middleware to verify JWT role claim and redirect unauthorised access server-side			
		A2.4 — Role-based UI conditional rendering	Implement client-side role guards to show/hide navigation, action buttons, and dashboard sections by role			
		A2.5 — RLS policy unit testing	Write and execute cross-role test cases verifying each role cannot access data outside their boundary			
A3	Public Browsing (Announcements, Programme Brief)	A3.1 — Public announcements page	Build unauthenticated announcements listing page fetching published SFC announcements from PostgreSQL	NA	7	Sprint 1
		A3.2 — Training programme brief listing	Build public page displaying available Guide and Ranger training tracks per TPA with overview and eligibility info			
		A3.3 — Public route configuration & SEO	Configure public Next.js routes, metadata tags, and Open Graph data for search discoverability			

Track B — Training & Certification Management

B1	Training Module Upload & Management (HoD)	B1.1 — Module CRUD API routes	Implement Next.js API routes for create, read, update, delete on training modules with HoD role guard	A1, A2	9	Sprint 1
		B1.2 — Media file upload to Supabase Storage	Implement file upload for training videos, PDFs, and images to Supabase Storage with size validation			
		B1.3 — Module editor UI (HoD dashboard)	Build HoD module editor with rich text input, file attachment, TPA selector, and publish/draft toggle			
		B1.4 — TPA-specific module categorisation	Tag each module to a specific TPA and certification track; filter module listings by TPA on guide views			
		B1.5 — Module versioning & update history	Track module edit history with timestamps and editor identity in PostgreSQL for audit and rollback			
B2	Training Module View & Completion (Guide)	B2.1 — Module listing with progress indicator	Build guide-facing module listing showing assigned TPA modules with completion percentage and locked state	B1	9	Sprint 1
		B2.2 — Content player component	Implement video player and PDF viewer components for consuming module content within the platform			
		B2.3 — Progress state persistence	Record and update guide progress per module in PostgreSQL on content consumption milestones			
		B2.4 — Module completion marking	Trigger completion state on all content consumed; update guide badge eligibility flag in PostgreSQL			
B3	Quiz System	B3.1 — Quiz builder (HoD)	Build HoD quiz builder with multiple-choice question editor, correct answer selection, and per-module assignment	B1	8	Sprint 1
		B3.2 — Quiz attempt UI (Guide)	Build guide quiz interface with question navigation, answer selection, timer support, and submit confirmation			
		B3.3 — Result scoring & storage	Calculate and persist quiz score in PostgreSQL on submission; flag pass/fail based on configurable threshold			
		B3.4 — Retry and result display logic	Show result summary after submission; allow retry on failure with configurable attempt limit; notify HoD on pass			
B4	Certification Application Submission	B4.1 — Application form UI	Build certification application form collecting park selection, training completion evidence, and document uploads	B2	9	Sprint 1
		B4.2 — Application submission API route	Implement submission endpoint validating training completion prerequisites before inserting to PostgreSQL			
		B4.3 — Application status tracking dashboard	Show guide their current application status (pending, under review, interview scheduled, approved, rejected)			
		B4.4 — Supporting document upload	Allow guides to attach supporting documents to their application via Supabase Storage			
B5	HoD Certification Review & Interview Scheduling	B5.1 — Application review dashboard (HoD)	Build HoD dashboard listing all pending applications with guide profile, training results, and quiz scores	B4	9	Sprint 1
		B5.2 — Approve/reject action with reason	Implement approve and reject actions with mandatory reason field; update application status in PostgreSQL			
		B5.3 — Interview scheduling form	Build interview scheduling UI for HoD to set date, time, and location; persist record linked to application			
		B5.4 — Email notification on decision	Trigger Supabase email on approval, rejection, and interview scheduling events with relevant details			

B6	Stripe Payment Integration (Sandbox)	B6.1 — Stripe SDK setup & environment config	Install Stripe Node.js SDK, configure sandbox API keys in environment variables, and initialise client server-side	B4	7	Sprint 1
		B6.2 — Payment session creation API route	Implement API route to create Stripe Checkout Session for certification fee on enrolment confirmation			
		B6.3 — Stripe webhook handler	Implement webhook endpoint to receive payment succeeded and payment failed events and update enrolment status			
		B6.4 — Payment failure retry flow	Implement retry UI allowing guide to reattempt payment with configurable retry limit before cancellation			
		B6.5 — Payment status display	Show payment status badge on guide application dashboard; update in real time on Stripe webhook receipt			
B7	Badge/Certificate Issuance & Email Delivery	B7.1 — Badge record insertion to PostgreSQL	Insert issued badge record with park, track, guide ID, issue date, and renewal due date to PostgreSQL	B5, B6	9	Sprint 1
		B7.2 — Certificate PDF generation	Generate certificate PDF with guide name, park name, certification track, and issue date server-side			
		B7.3 — Email delivery with certificate attachment	Send certificate PDF as email attachment to guide via Supabase email trigger on badge issuance			
		B7.4 — Badge display on guide profile	Show active badges on guide profile with park name, track, issue date, expiry, and renewal status			
B8	Senior Auto-Nomination Logic	B8.1 — Badge renewal history check (Edge Function)	Write Supabase Edge Function to periodically query badge renewal records and identify guides meeting Senior criteria	A2, B7	6	Sprint 1
		B8.2 — Nomination criteria configuration	Allow SFC Management to configure the qualifying badge renewal period threshold for Senior nomination			
		B8.3 — Nomination notification to HoD	Trigger in-app and email notification to HoD when a guide is auto-nominated for Senior promotion			
		B8.4 — Senior role promotion on confirmation	Update user role from GUIDE to SENIOR_GUIDE in Supabase Auth and PostgreSQL on HoD approval			
Track C — Notifications & Communication						
C1	Notification System — Email	C1.1 — Supabase email trigger configuration	Configure Supabase Auth and database email triggers for certification events, interview scheduling, and badge issuance	A2	8	Sprint 1
		C1.2 — Email template design per notification type	Design HTML email templates for each notification type (approval, rejection, interview, badge, welcome)			
		C1.3 — Email delivery logging	Log email dispatch events (type, recipient, timestamp, status) to PostgreSQL for audit and verification			
C2	Notification System — SMS (Twilio)	C2.1 — Twilio SDK setup & environment config	Install Twilio Node.js SDK, configure Account SID, Auth Token, and phone number in environment variables	A2	7	Sprint 1
		C2.2 — SMS API route with message templates	Implement Next.js API route accepting notification type and phone; construct and dispatch SMS via Twilio REST API			
		C2.3 — SMS delivery status logging	Log Twilio message SID, delivery status, and timestamp to PostgreSQL for monitoring and failure detection			
C3	Notification System — In-App (Supabase Realtime)	C3.1 — Supabase Realtime subscription setup	Configure Supabase Realtime channel subscriptions on the notifications table filtered by authenticated user ID	A2	8	Sprint 1
		C3.2 — Notification bell UI component	Build notification bell icon with unread count badge and dropdown listing recent notifications			
		C3.3 — Read/unread state management	Implement mark-as-read on notification click; persist read state in PostgreSQL; update bell count reactively			

C4	Notification System — PWA Push (Web Push API)	C4.1 — VAPID key generation & server config	Generate VAPID key pair, store private key in server environment, and expose public key to client	A2	7	Sprint 2
		C4.2 — Push subscription management API	Implement subscribe and unsubscribe API routes; persist push subscription objects in PostgreSQL per device			
		C4.3 — Push notification dispatch logic	Implement server-side Web Push dispatch using web-push library; trigger on relevant system events			
		C4.4 — Service worker push event handler	Write service worker push event listener to display system notification with title, body, and icon on device			
Track D — AI Monitoring System						
D1	PWA Service Worker Setup & Model Caching	D1.1 — Service worker registration & lifecycle	Register service worker in Next.js with install, activate, and fetch lifecycle handlers; configure scope and update strategy	A1	9	Sprint 1
		D1.2 — Model file cache-first strategy	Configure service worker to fetch and cache the AI model file on install; serve from cache on all subsequent requests			
		D1.3 — Cache invalidation & model update	Implement versioned cache key strategy; fetch updated model on new deployment and replace cached version			
		D1.4 — Offline asset caching configuration	Define cache list for app shell assets (JS, CSS, fonts, icons) to enable full offline PWA functionality after first load			
D2	Device Camera Initialisation & Live Feed	D2.1 — getUserMedia permission & error handling	Implement camera permission request with fallback messaging for denied, unavailable, or HTTPS-missing scenarios	D1	9	Sprint 2
		D2.2 — Video stream render to canvas	Pipe live camera stream to HTML canvas for frame extraction; configure resolution and frame rate constraints			
		D2.3 — Session start/stop controls	Build monitoring session UI with Start and Stop controls; manage camera stream and inference loop lifecycle			
		D2.4 — Frame extraction for inference input	Implement requestAnimationFrame loop extracting canvas frames at configurable interval as inference input tensors			
D3	On-Device Inference (TensorFlow.js / ONNX)	D3.1 — TensorFlow.js / ONNX Runtime Web integration	Install and configure runtime; benchmark both on target device and select based on FPS performance	D1, D2	10	Sprint 2
		D3.2 — Model loading from service worker cache	Implement model load from Cache Storage API using cached model URL; handle load failure with retry			
		D3.3 — Frame-by-frame inference loop	Run inference on extracted frames; output bounding boxes, class labels, and confidence scores per frame			
		D3.4 — Confidence threshold evaluation	Filter inference outputs by configurable threshold (default 70%); map class label to prohibited activity type			
D4	On-Device Alert (Sound & Visual)	D4.1 — Audio alert on detection trigger	Play pre-loaded alert sound via Web Audio API immediately on detection without network dependency	D3	8	Sprint 2
		D4.2 — Visual alert overlay component	Display full-screen alert overlay showing detected activity type, confidence score, and timestamp			
		D4.3 — Alert dismiss & session continuation	Allow guide/ranger to dismiss alert and continue monitoring; log alert acknowledgement timestamp			

D5	Evidence Capture & H.264 Compression	D5.1 — Clip capture from live stream on detection	Capture a configurable-duration clip from the canvas stream into a Blob on detection trigger	D3	9	Sprint 2
		D5.2 — FFmpeg.wasm integration & H.264 encoding	Load FFmpeg.wasm, write captured Blob to virtual filesystem, execute H.264 re-encode with configurable CRF setting			
		D5.3 — Timestamp & metadata attachment	Record detection timestamp, GPS coordinates, guide ID, activity type, and confidence score with the clip			
		D5.4 — Compressed output size validation	Verify compressed output is within acceptable bounds before queuing for upload; log compression ratio			
D6	Online Upload — Supabase Storage + PostgreSQL	D6.1 — Supabase Storage clip upload	Upload compressed H.264 clip to Supabase Storage evidence bucket with content-type video/mp4 and access policy	D5	9	Sprint 2
		D6.2 — PostgreSQL metadata insert	Insert evidence record with clip URL, guide ID, activity type, confidence, GPS, and timestamp to evidence_logs table			
		D6.3 — Upload progress indicator UI	Show upload progress bar during clip transmission; display success or failure state on completion			
		D6.4 — Upload error handling & retry	Catch upload failures; move failed uploads to offline queue for retry on reconnect			
D7	Offline Queue & Background Sync	D7.1 — Local queue implementation (IndexedDB)	Implement IndexedDB-backed queue to store compressed clips and metadata when device is offline at detection time	D5, D6	8	Sprint 2
		D7.2 — navigator.onLine event listener	Register online/offline event listeners; trigger background sync attempt on transition from offline to online			
		D7.3 — Background sync execution	On reconnect, iterate queue and upload all pending clips in order; remove each item on successful upload			
		D7.4 — Queue status display on UI	Show pending sync count badge on monitoring dashboard; display last sync timestamp and queue depth			
Track E — Administration, QA & Delivery						
E1	Admin Evidence Review Dashboard	E1.1 — Evidence clip listing with filter & search	Build HoD dashboard listing all evidence clips with filter by park, guide, activity type, and date range	D6	8	Sprint 2
		E1.2 — In-platform video player	Implement H.264 video player rendering stored evidence clips via Supabase Storage signed URL			
		E1.3 — Metadata display panel	Show clip metadata (guide, park, activity type, confidence, GPS, timestamp) alongside video player			
		E1.4 — Evidence export & download	Allow HoD to download evidence clips and export metadata records as CSV for reporting			
E2	Senior Guide Group Monitoring View	E2.1 — Group-scoped evidence clip listing	Build Senior Guide dashboard listing only evidence clips belonging to their assigned group, enforced by RLS	A2, D6	7	Sprint 2
		E2.2 — Training progress overview per guide	Show training completion percentage and certification status for each guide in the senior's assigned group			
		E2.3 — Group assignment management (HoD)	Allow HoD to assign guides to senior group leaders in admin dashboard; persist group membership in PostgreSQL			
E3	PWA Full Configuration (Offline, Installable)	E3.1 — Web app manifest configuration	Create manifest.json with app name, icons, theme colour, display mode (standalone), and start URL	A1, D1	7	Sprint 2
		E3.2 — Install prompt & home screen icon	Implement beforeinstallprompt event handler for custom install banner; configure 192px and 512px app icons			
		E3.3 — Offline asset caching & fallback page	Define service worker cache for critical app shell assets; implement offline fallback for uncached navigation			

E4	Vulnerability Assessment & Remediation	E4.1 — Testing scope & tools definition	Define SSDLC assessment scope covering API routes, RLS policies, authentication flows, and file upload endpoints	ALL	9	Sprint 2
		E4.2 — OWASP ZAP automated scan execution	Run OWASP ZAP active scan against staging deployment; capture and document all flagged vulnerabilities			
		E4.3 — Manual penetration testing	Perform cross-role access testing, JWT manipulation, SQL injection probing, and XSS testing on input fields			
		E4.4 — Vulnerability findings report	Document all findings with description, affected endpoint, severity (Critical/High/Medium/Low), and remediation plan			
		E4.5 — Remediation implementation	Implement fixes for all Critical and High severity findings; document changes made per vulnerability			
		E4.6 — Reassessment & sign-off report	Re-run assessment tools post-remediation; produce reassessment report confirming resolution of all vulnerabilities			
E5	User Manuals (Guide + Admin)	E5.1 — Guide manual — registration to certification	Document step-by-step registration, training module access, quiz completion, and certification application	ALL	6	Sprint 2
		E5.2 — Guide manual — AI monitoring session	Document how guides start a monitoring session, what alerts look like, and how to handle offline queuing			
		E5.3 — Admin manual — module & badge management	Document HoD workflows for creating modules, reviewing applications, scheduling interviews, and issuing badges			
		E5.4 — Admin manual — evidence review dashboard	Document how HoD accesses, filters, views, and exports evidence recordings from the admin dashboard			
		E5.5 — Manual review & screenshot verification	Review both manuals against staging; verify all screenshots match current UI; update any outdated steps			
E6	Final Testing, Bug Fixes & Deployment	E6.1 — End-to-end test execution across all tracks	Execute structured test cases covering all primary user flows across Track A–F; record pass/fail outcomes	ALL	9	Sprint 2
		E6.2 — Bug triage & fix prioritisation	Log all test failures; triage by severity; assign critical and high priority bugs for immediate resolution			
		E6.3 — Staging environment deployment verification	Verify all environment variables in Vercel staging; confirm all API integrations functional on staging URL			
		E6.4 — Final submission checklist sign-off	Complete checklist verifying all 29 backlog items are implemented and all deliverables are present			
Track F — Multilingual Support (Google Translate API)						
F1	Multilingual Support (Google Translate API)	F1.1 — Google Translate API setup & key config	Enable Google Cloud Translation API, generate API key, and configure in server-side environment variables	B1, B2	6	Sprint 2
		F1.2 — Language preference setting (user profile)	Add language preference field to user profile; build language selector UI on profile settings page			
		F1.3 — Server-side translation API route	Implement Next.js API route accepting text and target language; call Google Translate API and return translated text			
		F1.4 — Training module content translation	Apply translation to module titles, descriptions, and quiz questions on guide views based on language preference			
		F1.5 — Notification message translation	Apply translation to email, SMS, and in-app notification bodies before dispatch based on recipient preference			
		F1.6 — Translation cache & quota management	Cache translated strings in PostgreSQL keyed by content hash + language; monitor API usage against free tier limit			

SOLUTION DIRECTION

Following evaluation of four alternative approaches against knowledge, skill, and technology dimensions, the team selected a custom-built Next.js/Supabase fullstack application as the most appropriate solution. This decision was driven primarily by the absence of any existing platform that satisfies the combined requirements of park-specific certification management and offline-capable AI field monitoring, and by the team's existing proficiency with the selected stack. The solution direction was evaluated using a KoST framework, assessing Knowledge, Skill, and Technology dimensions as follows. The chosen solution is a custom-built, fullstack web application developed using **Next.js** as the frontend and backend framework, **Supabase** for authentication, database, storage, and real-time functionality, and deployed via **Vercel**. The mobile experience is delivered through a **Progressive Web App (PWA)**, allowing guides and rangers to install the platform on their devices without App Store distribution. The AI monitoring component uses a pre-trained model hosted on the server and cached on-device via the PWA service worker. Inference runs locally on the device using the cached model and the device camera feed, with evidence compressed and synced to Supabase Storage.

SOLUTION 1 — CUSTOM-BUILT NEXT.JS / SUPABASE FULLSTACK PWA WITH ON-DEVICE AI

A purpose-built web and mobile platform developed using Next.js as the fullstack framework, Supabase for authentication, database, storage, and real-time services, and deployed via Vercel. The mobile experience is delivered through a Progressive Web App. The AI monitoring component uses a pre-trained computer vision model served from the server, cached on-device via the PWA service worker, and executed locally using TensorFlow.js — enabling offline inference without cloud dependency.

SOLUTION 2 — OFF-THE-SHELF LMS (MOODLE / TALENTLMS) WITH CLOUD-BASED AI INTEGRATION

An existing Learning Management System deployed or subscribed to for training delivery, with a separate cloud-based AI inference service (e.g. AWS Rekognition or Google Vision AI) integrated for field monitoring. The LMS would handle training modules and progress tracking; the AI component would receive video frames from a field device and return detection results via API.

KOST GAP ANALYSIS

Solution 1 — Custom Next.js / Supabase PWA

Team Member	Role	Knowledge Gap	Skill Gap	Technology Gap
Terence Chin Seng Wong	Project Lead, Full Stack Lead, AI & Media Engineer	None — has engaged directly with SFC to understand the training workflow, badge system, and field monitoring requirements; familiar with fullstack architecture and on-device AI integration patterns	None — has prior experience with Next.js, Supabase schema design, RLS policy authoring, and CI/CD via Vercel; leads architecture decisions across both application and AI tracks	None — the full stack (Next.js, Supabase, Vercel) and AI tooling (TensorFlow.js, FFmpeg.wasm) are within existing competency
Tammy Ru Xiu Tay	Full Stack Developer, AI & Media Engineer	Low — familiar with fullstack development and general AI integration concepts; service worker model caching and WebAssembly inference pipeline are newer areas	Low — competent in Next.js and Supabase; FFmpeg.wasm compression pipeline and TensorFlow.js in-browser runtime require additional learning	Low — PWA service worker caching and ONNX/TensorFlow.js are well-documented open-source tools; no major technology gap
Miaw Fong Lim	Frontend Developer	Low — understands React component structure and UI patterns; less familiar with role-gated routing in Next.js App Router and PWA manifest configuration	Low — competent in React and Tailwind CSS; Next.js middleware for route protection and service worker setup are new but approachable	Low — PWA-specific configuration and Web Push API are new but well-documented; no fundamental technology gap
Emily Hui Xiu Jong	Frontend Developer	Low — familiar with React and dashboard UI patterns; less familiar with Supabase Realtime WebSocket integration for live in-app notifications	Low — competent in React; Supabase Realtime subscription patterns and role-specific view rendering in Next.js are new areas	None — UI frameworks and frontend tooling are fully within existing competency

Zi Yang Lim	Full Stack Developer, AI & Media Engineer	Medium — familiar with fullstack development; TensorFlow.js in-browser inference, ONNX Runtime Web, and WebAssembly-based ML execution are new knowledge domains	Medium — FFmpeg.wasm compression pipeline and on-device inference setup require significant new skill acquisition; offline queue and background sync patterns are also new	Low — TensorFlow.js, ONNX Runtime, and FFmpeg.wasm are open-source with active communities; integration patterns are well-documented despite being new to the team
Young Fung Lau	Backend Developer, QA & Security Engineer	Medium — familiar with REST API development and general testing principles; Supabase Edge Functions, Stripe webhook handling, and SSDLC-structured vulnerability assessment methodology are new domains	Medium — Stripe integration, serverless Edge Function patterns, and PostgreSQL RLS policy authoring require learning; security scanning tools (e.g. OWASP ZAP) are new	Low — Stripe, Supabase, and OWASP tooling are mature and well-documented; no specialist technology gap
Brian Jia Hua Liew	Backend Developer, QA & Security Engineer	Medium — familiar with API development; Twilio SMS delivery, Web Push VAPID key authentication, and OTP flows via Supabase Auth are new knowledge areas	Medium — Twilio and Web Push API integration patterns are new; structured test case writing and RLS policy verification require learning	Low — Twilio, Web Push API, and Supabase Auth are well-documented third-party SDKs; Lighthouse and browser DevTools for audit and verification are accessible

Solution 2 — Off-the-Shelf LMS (Moodle/TalentLMS) with Cloud-Based AI Integration

Team Member	Role	Knowledge Gap	Skill Gap	Technology Gap
Terence Chin Seng Wong	Project Lead, Full Stack Lead, AI & Media Engineer	High — Moodle/TalentLMS plugin architecture and administration are unfamiliar; integrating a cloud AI vision API with an LMS requires understanding two entirely separate platform architectures simultaneously	High — LMS customisation uses PHP-based plugin systems and platform-specific templating; cloud AI streaming (frame capture to REST API) is architecturally different from on-device inference	Medium — both LMS platforms and cloud vision APIs exist and are documented, but the integration pattern between them is non-standard and outside existing competency
Tammy Ru Xiu Tay	Full Stack Developer, AI & Media Engineer	High — LMS administration and cloud AI service configuration are entirely new domains; no prior exposure to Moodle plugin development or AWS/Google Vision API	High — cloud vision API integration requires understanding of frame streaming, API latency handling, and result processing — none of which map to existing fullstack experience	Medium — AWS Rekognition and Google Vision AI are mature platforms but require platform-specific SDK knowledge the team does not currently hold
Miaw Fong Lim	Frontend Developer	High — Moodle frontend customisation uses Mustache templates and PHP rendering, not React; the entire frontend skill set would not transfer to this solution	High — no experience with LMS theme development or PHP-based UI customisation; the technology stack is fundamentally different from existing competency	High — completely different frontend stack; React and Tailwind knowledge cannot be applied to LMS customisation
Emily Hui Xiu Jong	Frontend Developer	High — same as Miaw; LMS frontend development requires platform-specific knowledge entirely outside current React-based experience	High — same as Miaw; Moodle/TalentLMS UI development requires PHP and platform-specific templating with no transferable skill from current stack	High — same as Miaw; the technology gap is fundamental, not bridgeable through documentation alone within the sprint timeline

Zi Yang Lim	Full Stack Developer, AI & Media Engineer	High — cloud AI inference (AWS Rekognition, Google Vision) is conceptually simpler than on-device inference but requires understanding of LMS integration architecture on top; LMS plugin development is an entirely new domain	Medium — calling a cloud vision REST API is within general backend competency; however, building the integration bridge between LMS and cloud AI requires platform-specific knowledge	Medium — cloud vision APIs are well-documented; the gap is in LMS integration architecture rather than the AI tooling itself
Young Fung Lau	Backend Developer, QA & Security Engineer	High — LMS backend customisation requires PHP plugin development knowledge; vulnerability assessment scope expands significantly to include the LMS platform's own security surface	High — LMS-specific security testing and plugin development require well beyond standard web application assessment	High — OWASP guidance for LMS platforms differs substantially from custom web application testing; Moodle plugin architecture is a specialised technology stack
Brian Jia Hua Liew	Backend Developer, QA & Security Engineer	High — LMS administration and plugin integration patterns are entirely new; cloud AI API authentication and rate-limiting handling are also new domains	High — no prior experience with LMS plugin systems or cloud vision API integration; test coverage for an LMS customisation introduces a different testing surface	High — the combination of LMS plugin architecture and cloud vision API creates two simultaneous high technology gaps with no existing team foundation

PROPOSED SOLUTION DIRECTION AND JUSTIFICATION

Following the KoST gap analysis across both solutions, Solution 1 — the custom-built Next.js/Supabase PWA with on-device AI — is the proposed solution direction.

The gap analysis reveals that Solution 2 introduces High-rated gaps across the majority of the team, particularly in knowledge and skill dimensions. The LMS platform requires a PHP-based customisation stack that no team member has prior experience with, and the cloud AI integration introduces a connectivity dependency that is fundamentally incompatible with SFC's remote park environments — a constraint that cannot be mitigated through skill acquisition alone.

Solution 1's gaps are concentrated in two areas: the AI and media engineering track, and the third-party backend integrations. These are addressed through the following mitigations:

- Tammy, Zi Yang, and Terence's medium-rated gaps in TensorFlow.js, ONNX Runtime, and FFmpeg.wasm are manageable within the sprint timeline given the availability of comprehensive open-source documentation and active community resources. Backlog item D1 (service worker setup and model caching) is deliberately placed in Sprint 1 to allow early prototyping and familiarisation before the full inference pipeline is built in Sprint 2.
- Young Fung and Brian's medium-rated gaps in Stripe, Twilio, Supabase Auth, and Web Push API integration are mitigated by the fact that all four services provide well-documented SDKs with extensive tutorials and active developer communities — the learning curve is present but time-bounded and does not require the team to operate outside their existing backend development discipline.

No team member faces a technology gap for Solution 1 that cannot be bridged through documentation and early prototyping. Solution 2, by contrast, presents High-rated technology gaps — specifically LMS plugin architecture and cloud vision API integration — that would require the entire team to operate outside their existing competency domain for the majority of deliverables, making it unsuitable within the two-sprint timeline.

DISCARDED POTENTIAL ALTERNATIVE SOLUTION

#	Alternative	Knowledge	Skill	Technology	Reason Discarded
1	Native Mobile App (React Native / Flutter)	High	Low	Exists	Requires separate iOS and Android codebases, significantly increasing development effort for a team of 8 within a 2-sprint timeline. PWA achieves equivalent mobile accessibility without the overhead.
2	Cloud-based AI Inference (e.g. AWS Rekognition, Google Vision AI)	Low	Low	Exists	Client use case has unreliable internet connectivity. A cloud-dependent AI system would fail in offline conditions, making it unsuitable for field deployment.
3	Firebase + React stack	High	Medium	Exists	Firebase lacks native Row Level Security comparable to Supabase's PostgreSQL RLS, making role-based data access control more complex to implement correctly. Supabase also provides built-in Auth, Storage, and Edge Functions as a unified platform, reducing integration overhead.

Table 7 – Discarded potential solution

AI MONITORING — EDGE DEVICE FEASIBILITY EVALUATION

The AI monitoring component requires a device capable of running real-time computer vision inference against a live camera feed. Four candidate approaches were evaluated: cloud-based inference, laptop or desktop browser, Raspberry Pi, and Arduino. The chosen approach will be confirmed and documented in the Sprint 1 report following hands-on benchmarking.

- Cloud-Based Inference

In this model, video frames are captured on the field device and transmitted to a cloud server (e.g., AWS Rekognition, Google Vision AI, or a custom server running YOLO) where inference is performed, with results returned to the device.

Pros	No compute constraint on the field device; model updates are instant server-side; highest possible model complexity
Cons	Requires continuous internet connectivity — unusable in remote park environments with unreliable signal; introduces latency between capture and alert; raises privacy concerns as raw video frames leave the device
Constraint	Fundamentally incompatible with SFC's field deployment context
Verdict	Discarded — connectivity dependency makes this unsuitable for any TPA with poor signal coverage

- Laptop / Desktop Browser (PWA with Service Worker Model Caching)

The pre-trained model is hosted on the server and downloaded on first PWA load, then cached by the service worker. Inference runs in-browser using TensorFlow.js or ONNX Runtime Web (WebAssembly). The device camera is accessed via `MediaDevices.getUserMedia`.

Pros	No specialist hardware required; model runs fully offline after initial cache; works on any PWA-capable device; compression (FFmpeg.wasm) and upload can run in the same browser context; easiest to develop, test, and demonstrate in an assignment context
Cons	Browser-based WebAssembly inference is slower than native inference on equivalent hardware; performance is device-dependent; iOS Safari has historically imposed stricter service worker storage limits which may affect model caching
Constraint	Inference speed at acceptable frame rates (≥ 15 FPS) depends on the host device's CPU/GPU. Lightweight models (MobileNet, YOLOv8n) are well-suited; heavier models may be impractical
Verdict	Proposed approach for this project — balances offline capability, development accessibility, and deployment simplicity within the assignment constraints

- Raspberry Pi (Pi 4 Model B or Pi 5)

A single-board computer running a Python-based inference stack (e.g., OpenCV + TensorFlow Lite or ONNX Runtime). Camera input via Raspberry Pi Camera Module or USB webcam. The Pi would run as a standalone embedded device in the field.

Pros	Native inference is significantly faster than browser-based WebAssembly; TensorFlow Lite is optimised for ARM; the Pi 4/5 can achieve real-time inference on lightweight models at 20–30 FPS; headless operation possible; lower power consumption than a laptop
Cons	Requires physical hardware procurement and setup; Python inference stack is separate from the Next.js web platform — integration requires a local API bridge; less practical for an assignment demonstration context; adds hardware dependency and cost
Constraint	Pi 4 (4GB RAM) with TensorFlow Lite can run YOLOv8n at approximately 10–20 FPS depending on input resolution. Pi 5 improves on this. Hardware availability and cost may be a limiting factor
Verdict	Technically viable and worth considering for a production SFC deployment — a Pi mounted on a body-worn camera harness would be more field-appropriate than a laptop.

- **Arduino (e.g., Arduino Nicla Vision, Arduino Nano 33 BLE Sense)**

Microcontroller-based inference using TinyML frameworks (Edge Impulse, TensorFlow Lite for Microcontrollers). The Arduino Nicla Vision includes an onboard camera and microcontroller capable of basic image classification.

Pros	Extremely low power consumption; compact and wearable; very low cost; purpose-built for embedded IoT sensing
Cons	Severely limited compute — Arduino microcontrollers have kilobytes of RAM and cannot run object detection models of meaningful complexity; limited to very simple classification tasks; not capable of running YOLOv8 or similar models; no browser context, so evidence upload requires a separate communication layer (Bluetooth, WiFi module)
Constraint	The Arduino Nicla Vision has 1MB Flash and 64KB RAM — insufficient for any multi-class object detection model. Only basic binary classification (activity vs no activity) is feasible, which is insufficient for SFC's multi-activity detection requirement
Verdict	Not suitable for this use case — compute constraints prevent deployment of any model complex enough to detect the range of prohibited activities required

Browser-based inference via PWA service worker caching is the proposed approach for this project, chosen for its development accessibility and zero hardware dependency. The Raspberry Pi remains the strongest candidate for a production SFC deployment and will be evaluated further in the Sprint 1 report once inference benchmarking has been conducted on available hardware.

HIGH-LEVEL SYSTEM ARCHITECTURE

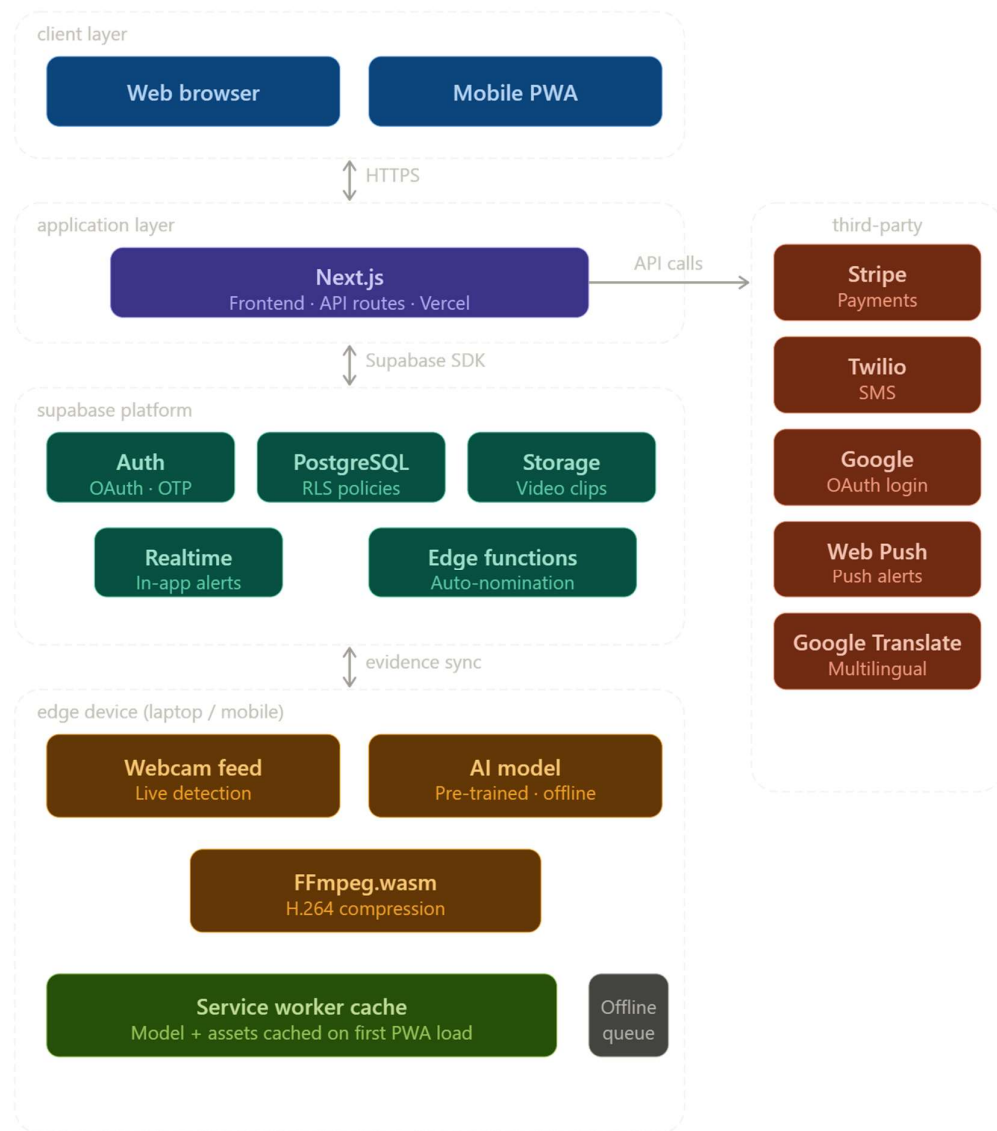


Figure 2 - High level system architecture diagram

System Architecture

The platform is structured across five distinct tiers as the figure above shows, each with a clearly defined responsibility. Together they form a cohesive system that handles user-facing interactions, business logic, data persistence, third-party integrations, and offline-capable AI field monitoring.

Tier 1 - Client Layer

The client layer is the entry point for all users interacting with the platform. It consists of two access modes that share the same underlying codebase and UI:

- Web Browser serves users accessing the platform through a standard desktop or laptop browser. This is the primary interface for HoD administrators managing training modules, reviewing certification applications, and accessing the evidence dashboard.
- Mobile PWA (Progressive Web App) allows guides and rangers to install the platform directly onto their mobile device home screen without requiring App Store or Play Store distribution. The PWA is built on top of the same Next.js frontend and is configured with a web app manifest and service worker that handles offline caching of both application assets and the AI detection model. On first load, the service worker fetches and caches the model file from the server, enabling the device camera and on-device inference to function fully without a network connection thereafter and push notification support via the Web Push API. This is the primary access mode for field staff who need reliable access in low-connectivity environments.

Both clients communicate exclusively with the application layer over HTTPS, ensuring all data in transit is encrypted.

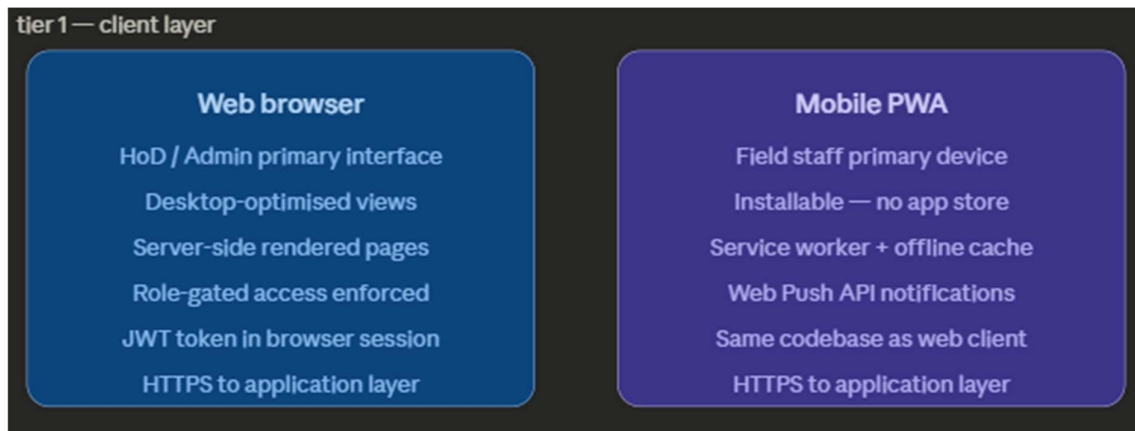


Figure 3 – Client Layer diagram

All requests originating from both client access modes are transmitted exclusively over HTTPS to the application tier, which serves as the sole gateway between the user-facing interface and all backend services — no client has direct access to the database, storage, or third-party APIs.

Tier 2 - Application Tier

The application layer is the central hub of the platform, built entirely on **Next.js** as a fullstack framework. It serves two responsibilities simultaneously:

- **Frontend rendering** — Next.js serves all UI pages for every role (Public, Guide, Ranger, Senior Guide, Senior Ranger, HoD, SFC Management). Role-specific views are enforced server-side, meaning users cannot access pages they are not authorised for regardless of client-side manipulation.
- **API Routes** — Next.js API routes handle all business logic including certification application processing, payment initiation via Stripe, notification dispatch via Twilio and Web Push, and communication with the Supabase platform. These routes run entirely server-side, keeping all credentials and sensitive operations away from the browser.

The application layer is deployed on **Vercel**, which provides automatic CI/CD on every push to the main branch, edge network distribution, and preview deployments for feature branches during development.

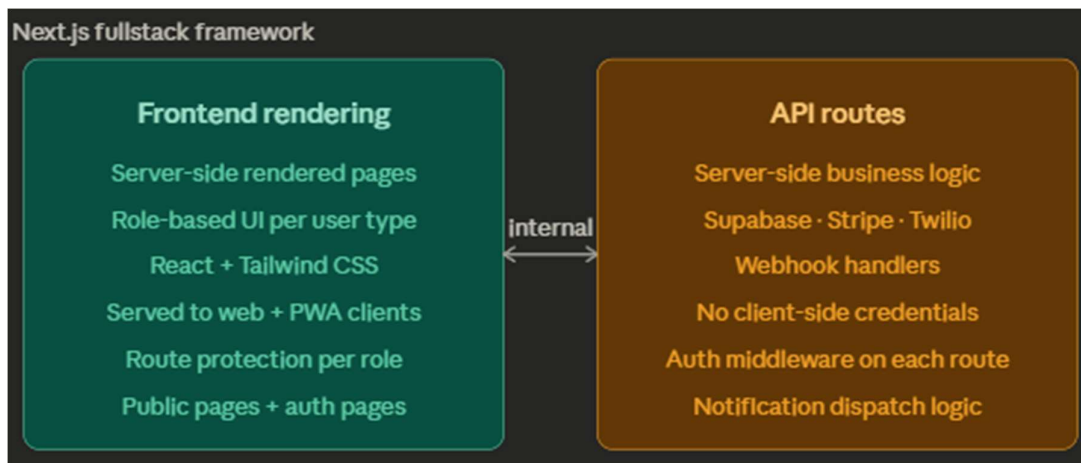


Figure 4 - Application Layer diagram

The application tier communicates with the Supabase platform via the Supabase SDK on the server side, delegating all data persistence, authentication state management, real-time event broadcasting, and background job execution to the services described below.

Tier 3 — Data Tier

Supabase serves as the unified backend infrastructure, consolidating five services under a single platform:

- **Auth** manages all user authentication. It supports two sign-in methods — Google OAuth for social login and OTP verification via email or phone number. Supabase Auth also issues and validates JWT tokens that carry the user's role, which all other Supabase services use to enforce access control.
- **PostgreSQL** is the primary relational database storing all structured data — user profiles, training records, certification applications, badge issuances, evidence metadata, and notification logs. Row Level Security (RLS) policies are defined at the database level, ensuring that each role can only read and write records they are authorised to access. For example, a guide can only query their own records, while a Senior Ranger can only see records belonging to their assigned group.
- **Storage** holds all binary assets uploaded to the platform, primarily the compressed H.264 evidence clips captured by the AI monitoring system. Each clip is stored with an access policy that restricts retrieval to authorised roles — guides cannot access other guides' clips, and senior staff can only access clips from their assigned group.
- **Realtime** powers the in-app notification channel. When a new notification record is inserted into the PostgreSQL database, Supabase Realtime broadcasts it instantly to the relevant user's active session over a WebSocket connection, delivering alerts without requiring the user to refresh the page.
- **Edge Functions** are serverless functions that run on Supabase's infrastructure, used for background tasks that should not block the main API. In this platform, Edge Functions handle the senior auto-nomination logic — periodically checking badge renewal history and promoting qualifying guides and rangers to Senior status automatically.

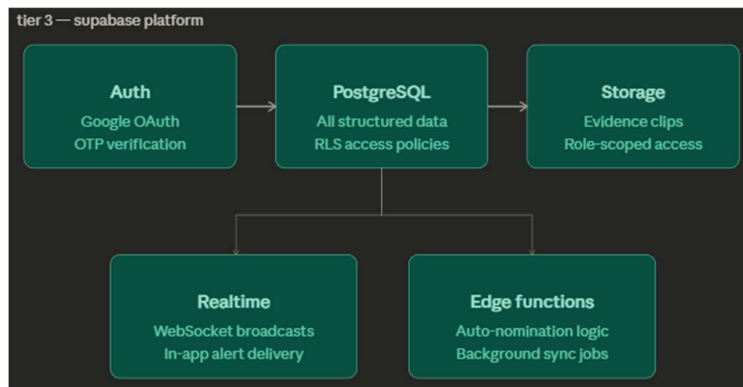


Figure 5 - Data Layer diagram

Where platform functionality requires capabilities beyond what Supabase provides natively — specifically payment processing, SMS delivery, and social authentication — the application tier makes outbound API calls to the following third-party service providers. All credentials for these services are held exclusively in server-side environment variables and are never exposed to the client.

Tier 4 — Third-Party Integrations

The application layer makes outbound API calls to five external service providers, each handling a specific function the platform does not implement natively:

- Stripe processes certification fee payments in sandbox mode. When a guide or ranger confirms enrolment in a certification programme, the application layer creates a Stripe payment session and redirects the user through Stripe's hosted checkout. On success, a webhook notifies the platform to proceed with enrolment confirmation.
- Twilio delivers SMS notifications for high-priority events where in-app or email alone is insufficient — such as interview scheduling, abnormal activity detection alerts, and certification outcomes. SMS is dispatched via a dedicated Next.js API route that constructs the message body based on the notification type before calling the Twilio REST API.
- Google OAuth is integrated through Supabase Auth to provide social login. When a user chooses to sign in with Google, Supabase manages the full OAuth 2.0 handshake, token exchange, and user profile creation without the platform needing to interact with Google's API directly.
- Web Push API enables push notifications to be delivered to PWA users even when the application is not open in the browser. The platform uses VAPID keys to authenticate push subscriptions, and notification payloads are sent server-side through the Web Push protocol when triggered by system events.

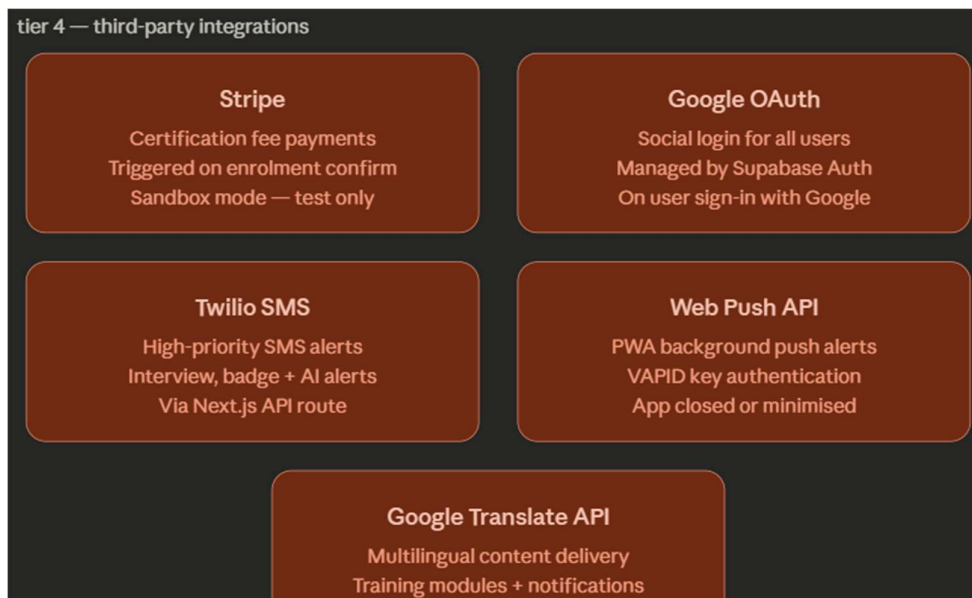


Figure 6 - Third-party Integration diagram

The edge device tier operates independently of all cloud infrastructure described above. While it communicates with the Supabase platform to upload evidence when connectivity permits, it is architecturally designed to function in full isolation — detection, compression, local storage, and on-device alerting all occur without any dependency on tiers 1 through 4 being reachable.

Tier 5 — AI Monitoring (On-Device Inference)

The AI monitoring tier is designed to function independently of network connectivity once the PWA has been initialised. It is not limited to a specific device — any PWA-capable device with a camera can serve as the monitoring endpoint. In the current prototype, a laptop or mobile device running the installed PWA serves this role.

- **Model serving and caching** — The pre-trained computer vision model is hosted on the server and registered as a service worker cache asset. When the PWA is first installed and opened, the service worker fetches the model file and stores it in the browser's cache storage alongside the application shell. On all subsequent sessions — including offline ones — the model loads directly from cache, with no server request required. This approach means the model is always available regardless of connectivity, and updates to the model can be pushed server-side and picked up automatically on the next online session.
- **Device camera feed** — Once a monitoring session is started, the device camera is initialised via the browser's MediaDevices.getUserMedia API. The live video stream is fed continuously into the cached model for frame-by-frame analysis.
- **On-device inference** — The model runs inference entirely within the browser using a WebAssembly-compatible ML runtime (such as TensorFlow.js). No video frames are transmitted to any server during analysis. This keeps inference latency low, preserves the privacy of recorded footage, and ensures the detection capability functions in areas with no network signal.
- **Evidence pipeline** — When a prohibited activity is detected above the confidence threshold, the system captures a timestamped clip, compresses it to H.264 using FFmpeg.wasm, then either uploads it immediately to Supabase Storage (if online) or queues it locally for background sync once connectivity is restored.
- **On-device alert** — An immediate sound and visual alert fires on the device at the moment of detection, independent of network status.

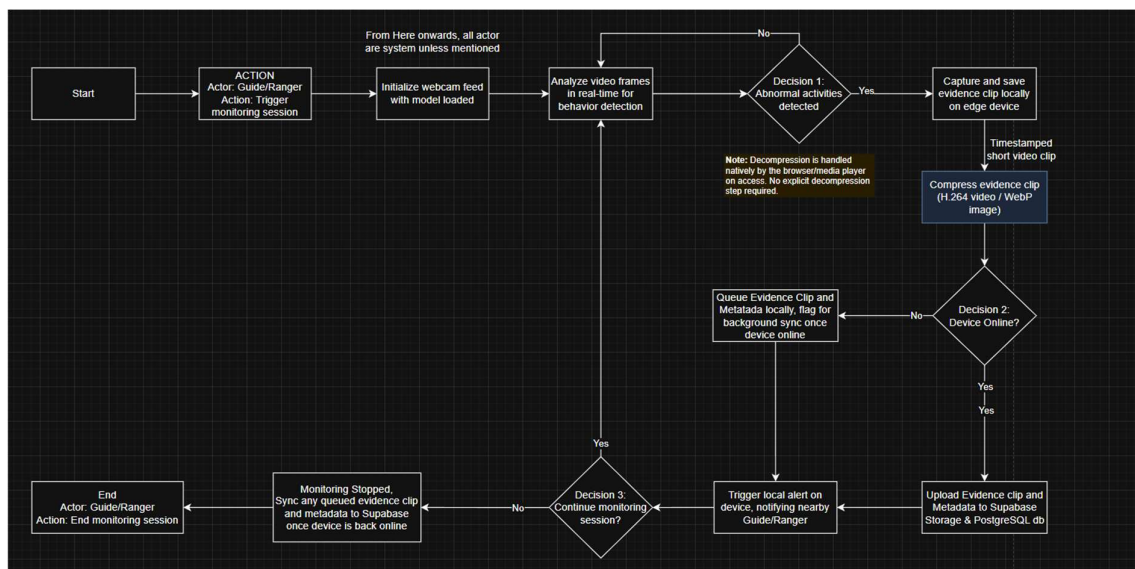


Figure 7 - AI Monitoring flow diagram

QUALITY MANAGEMENT

In the context of this project, quality is defined as the degree to which the delivered platform fulfils SFC's stated operational requirements reliably, securely, and accessibly — measured through functional correctness, security compliance, system performance, and user experience across all defined roles.

Quality is not measured by feature count alone. A platform that delivers all 29 backlog items but fails under concurrent users, exposes sensitive data through misconfigured access policies, or is too complex for a non-technical ranger to navigate independently does not meet the quality standard for this engagement. The following SMART quality criteria apply:

1. Functional Correctness

Specific: All 29 product backlog items must be implemented and demonstrable end-to-end without manual workarounds or hardcoded test bypasses.

Measurable: A structured test run covering every primary user flow (registration, training completion, certification application, badge issuance, AI detection, evidence upload) must pass with zero critical failures. A critical failure is defined as any failure that prevents a workflow from completing.

Achievable: Test cases will be written in Sprint 1 and executed in Sprint 2 prior to final deployment.

Relevant: SFC requires a platform that field staff and administrators can operate independently — any broken workflow directly undermines the platform's purpose.

Time-bound: All critical-path flows must pass testing by the end of Sprint 2, at least 3 days before the final submission deadline to allow time for fixes.

2. Security & Access Control

Specific: No user role must be able to access data, routes, or functionality outside their defined permission boundary, as specified by the role hierarchy in the Stakeholders section.

Measurable: Row Level Security (RLS) policies will be verified by attempting cross-role data access via both the application UI and direct Supabase API calls. Zero unauthorised data retrievals are acceptable — any policy bypass constitutes a security failure. The application vulnerability assessment must produce a reassessment report confirming all identified vulnerabilities have been remediated.

Achievable: RLS policies are defined at the database level in Supabase and are testable independently of the application code.

Relevant: The platform stores sensitive personal data, certification records, and video evidence — data leakage between roles would be a direct compliance and trust failure for SFC.

Time-bound: Initial RLS verification to be completed by the end of week 2 of Sprint 1. Vulnerability assessment and reassessment to be completed before the final week of Sprint 2.

3. AI Detection Reliability

Specific: The AI monitoring system must correctly detect defined prohibited activities and trigger the evidence capture pipeline without false-negative failures in a live demonstration environment.

Measurable: During acceptance testing, the system must successfully detect at least 3 distinct prohibited activity types with a confidence score at or above 70%, correctly capture and compress the evidence clip, and upload it to Supabase Storage within 60 seconds on a connected device. The offline queue must successfully sync at least one clip following simulated connectivity loss.

Achievable: Detection thresholds and compression parameters are configurable and will be tuned during Sprint 2 development.

Relevant: AI monitoring is one of the two core pillars of the platform — an unreliable detection system would undermine SFC's conservation enforcement objective entirely.

Time-bound: AI detection testing must be completed and documented by the end of Sprint 2 week 2, leaving a buffer for remediation before submission.

4. Performance & Accessibility

Specific: The platform must load within acceptable time thresholds and remain usable on standard mobile devices representative of those used by guides and rangers in the field.

Measurable: Initial page load on a 4G mobile connection must not exceed 5 seconds for the dashboard. The PWA must be installable and must serve cached content within 2 seconds when the device is offline. UI text and interactive elements must meet WCAG 2.1 Level AA contrast and sizing standards.

Achievable: Next.js server-side rendering and Vercel's edge network provide performance foundations. Lighthouse audits will be used as the measurement tool.

Relevant: Field staff in remote parks may be using older devices on degraded connections — a slow or inaccessible platform will simply not be used.

Time-bound: A Lighthouse audit must be run on the staging deployment by the final week of Sprint 2, with any scores below 70 (Performance, Accessibility) addressed before submission.

5. User Manual Completeness

Specific: Both user manuals (Guide/Ranger and HoD/Admin) must cover all primary workflows without requiring a reader to seek external assistance to complete a task.

Measurable: Each manual will be reviewed against a checklist of all primary workflows defined in the product backlog. Every workflow must be documented with step-by-step instructions and at least one supporting screenshot. Zero primary workflows may be undocumented.

Achievable: Manuals will be drafted in Sprint 2 once the UI is stable, using the working staging deployment for screenshots.

Relevant: The platform will be handed over to SFC for demonstration — administrators and guides must be able to operate the system independently without developer assistance.

Time-bound: Both manuals must be completed and reviewed internally by the end of Sprint 2 week 2.

RESOURCES

Hardware

Resource	Purpose	Provider
Development laptops (×7)	Individual development workstations for each team member	Team-owned
Edge device with camera(×1)	Dedicated device for AI monitoring development, webcam testing, and FFmpeg.wasm compression benchmarking	Team-owned
Mobile devices (iOS and Android)	PWA installation and caching testing, mobile UI validation, push notification testing	Team-owned
Webcam	Live video feed input for AI monitoring session testing	Team-owned

Software & Services

Development Stack

Resource	Purpose	Licence / Tier
Next.js	Fullstack framework — frontend rendering and API routes	Open source (MIT)
React	UI component library (included with Next.js)	Open source (MIT)
Tailwind CSS	Utility-first CSS framework for UI styling	Open source (MIT)
Node.js	JavaScript runtime environment	Open source (MIT)

Backend & Infrastructure

Resource	Purpose	Licence / Tier
Supabase	Auth, PostgreSQL database, Storage, Realtime, Edge Functions	Free tier
Vercel	Hosting, CI/CD pipeline, preview deployments	Free tier
GitHub	Version control, branch management, pull request workflow	Free tier

Third-Party Integrations

Resource	Purpose	Licence / Tier
Stripe	Certification fee payment processing (sandbox mode)	Free (sandbox)
Twilio	SMS notification delivery	Free trial
Web Push API + VAPID keys	PWA push notification delivery	Browser-native (no cost)
Google OAuth (via Supabase Auth)	Social login authentication	Free (Google Cloud)
Google Translate API	Multilingual content delivery for training modules and UI	Free tier

AI & Media Processing

Resource	Purpose	Licence / Tier
Pre-trained computer vision model (TBD — e.g. YOLOv8, MobileNet)	Abnormal activity detection — hosted on server, cached on-device via service worker	Open source (TBD)
TensorFlow.js / ONNX Runtime Web	In-browser WebAssembly ML runtime for on-device model inference	Open source (Apache 2.0)
FFmpeg.wasm	In-browser H.264 video compression of evidence clips	Open source (LGPL)
MediaDevices API	Browser-native device camera feed capture	Browser-native (no cost)

Development Tools

Resource	Purpose	Licence / Tier
Visual Studio Code	Primary code editor	Free
draw.io	System flow diagram creation	Free
Postman	API route testing and documentation	Free tier
npm	Package management	Free (open source)

PROJECT TEAM

Provide a summary of the team members and their roles.

	Name of student	Student Id	Signature	Roles
1	TERENCE CHIN SENG WONG	104404059	<i>WONG</i>	Project Lead, Full Stack Lead, AI & Media Engineer
2	TAMMY RU XIU TAY	104405861	<i>TAY</i>	Full Stack Developer, AI & Media Engineer
3	MIAW FONG LIM	104406013	<i>LIM</i>	Frontend Developer
4	EMILY HUI XIU JONG	102788832	<i>JONG</i>	Frontend Developer
5	ZI YANG LIM	102774129	<i>LIM</i>	Full Stack Developer, AI & Media Engineer
6	YOUNG FUNG LAU	102774585	<i>LAU</i>	Backend Developer, QA & Security Engineer
7	BRIAN JIA HUA LIEW	104381288	<i>LIAW</i>	Backend Developer, QA & Security Engineer

Project Sponsor [Dr. Lee Su Han]

Tutor's name (on behalf of the client)	Signature:
Ts, Dr. Lee Sue Han	